Trabajo práctico 2

Compresión de imágenes sin pérdida mediante Codificación de Huffman

1. Introducción

La compresión de datos busca representar la información utilizando una menor cantidad de bits. Desde el punto de vista de la teoría de la información, la eficiencia de una representación depende, entre otros factores, de las probabilidades de aparición de los símbolos y de la redundancia presente en los datos. En este trabajo se estudiarán técnicas de compresión **sin pérdidas** aplicadas a imágenes en escala de grises. En particular, se analizará el algoritmo de codificación de Huffman y se investigará cómo puede cómo impacta en su eficiencia la dependencia entre píxeles vecinos.

1.1. Reseña histórica

Antes del desarrollo formal de la teoría de la información, la compresión de mensajes era un problema abordado empíricamente, sin una formulación matemática. Un ejemplo temprano es el *Código Morse* (1838), diseñado por Samuel Morse y Alfred Vail para el telégrafo eléctrico [1]. Este sistema asignaba secuencias cortas a las letras más frecuentes y largas a letras poco comunes (por ejemplo, para el idioma Inglés, `E` → ▪ y `J` → ▪▬▬▬), anticipando el principio fundamental de lo que sería una codificación binaria eficiente: asignar menos bits (o señales) a los símbolos más probables. Con la publicación de Claude Shannon [2], surgió una base teórica para la codificación de la información. Shannon demostró que la entropía $H(X)$ de una fuente X representa el límite inferior teórico para la longitud promedio (en bits) de cualquier código sin pérdida de información. Poco después, junto a Robert Fano propusieron el método de *Shannon–Fano coding* [3], que construye un código de longitud variable para cada símbolo.

En 1951 David A. Huffman, alumno de Robert M. Fano, desarrolló el *código de Huffman* como parte de un trabajo práctico de la asignatura "teoría de la información" en el MIT. Para esta tarea, Huffman encontró el código binario más eficiente, creando un método basado en un árbol binario ordenado por frecuencias, el cual fue presentado en su artículo de 1952 [4]. Este algoritmo construye un árbol binario para la generación de un código libre de prefijos óptimo, bajo la restricción de que ningún código asignado sea prefijo[1] de otro (ver apéndice). El trabajo de Huffman marcó un hito en la teoría de la información aplicada, estableciendo el estándar de codificación óptima sin pérdida. A partir de este desarrollo, la disciplina evolucionó hacia nuevos paradigmas de compresión, tales como la codificación aritmética [5] y los algoritmos basados en diccionarios de Lempel y Ziv [6, 7]. No obstante, la codificación de Huffman no fue reemplazada por completo; por el contrario, sigue siendo un componente esencial de numerosos estándares modernos como ZIP (mediante el algoritmo DEFLATE), PNG, JPEG y FLAC, donde usualmente actúa como la etapa final de codificación debido a su enorme simplicidad y eficiencia teórica.

[1] El código de un símbolo se dice prefijo de otro si la cadena de bits que lo representa coincide con los primeros bits de la cadena que representa al otro símbolo. Ej: si $A \to 110$ y $B \to 11010$, entonces 110 es prefijo de 11010.

1.2. Compresión sin pérdida y codificación de Huffman

En la compresión **sin pérdida**, la secuencia original debe poder recuperarse exactamente a partir de su representación codificada. Aplicando el algoritmo de Huffman a una fuente de símbolos X, se puede lograr que la longitud media de codificación

$$L = \sum_x p(x)l(x),$$

donde $l(x)$ es la longitud del código asignado al símbolo x y $p(x)$ la función de masa de probabilidad. Para un código de Huffman binario se cumple $H(X) \leq L < H(X) + 1$, siendo la entropía $H(X)$ el límite inferior para la longitud media de esta codificación sin pérdida.

1.3. Codificación de imágenes píxel a píxel

Una imagen en escala de grises puede representarse mediante una matriz cuyos elementos indican la intensidad de cada píxel. Si la imagen es de 8 bits por píxel, cada intensidad es un entero entre 0 y 255. La representación directa utiliza 8 bits para cada píxel, independientemente de su frecuencia. Para aplicar la compresión por codificación de Huffman, una posibilidad es considerar cada intensidad X como un símbolo de la fuente. De esta manera, cada nivel de gris será representado con códigos más cortos o más largos dependiendo su probabilidad $p(x)$.

1.4. Codificación diferencial mediante residuos

En una imagen, los píxeles cercanos suelen tener intensidades similares. Esta dependencia espacial puede aprovecharse para obtener una representación alternativa, ya que muchas imágenes presentan redundancia espacial que puede utilizarse para mejorar la compresión. En lugar de codificar directamente cada píxel, podemos codificar la diferencia de intensidad entre un píxel y un píxel vecino [8]. Por ejemplo, si recorremos la fila i de izquierda a derecha, definimos el residuo

$$y_{i,j} = x_{i,j} - x_{i,j-1}, \qquad j = 2, \ldots, N,$$

donde $x_{i,j}$ es la intensidad del píxel ubicado en la fila i y la columna j, y N es el número de columnas de la imagen. Es importante tener en cuenta que debe guardarse la intensidad del primer píxel de cada fila, $x_{i,1}$, ya que no tiene un píxel anterior dentro de esa fila y se utiliza como condición inicial. De esta manera, la codificación de Huffman puede aplicarse sobre los residuos $y_{i,j}$ en lugar de hacerlo sobre los valores absolutos $x_{i,j}$ de cada píxel. Para reconstruir la imagen, primero se aplica la decodificación de Huffman y luego se recuperan los píxeles sumando sucesivamente cada residuo al píxel anterior reconstruido. Para ello, se utiliza el valor inicial $x_{i,1}$:

$$x_{i,j} = x_{i,j-1} + y_{i,j}, \qquad j = 2, \ldots, N.$$

Importante: para recorrer toda la imagen pueden aplicarse distintas estrategias. Una posibilidad es guardar el primer píxel de cada fila y calcular los residuos por fila. Otra es realizar un recorrido en forma de serpentina, alternando el sentido del recorrido en filas consecutivas, de modo que pueda guardarse un único píxel completo como inicial para toda la imagen y el resto solo residuos. En todos los casos, los residuos deben calcularse entre píxeles vecinos según el recorrido elegido. Por ejemplo, si se recorre cada fila de izquierda a derecha, no debe calcularse una diferencia entre el último píxel de una fila y el primero de la siguiente.

2. Ejercicios

En los siguientes ejercicios se trabajará con al menos tres imágenes provistas en el campus, aunque pueden agregarse otras que se consideren pertinentes para el análisis. Para la codificación y decodificación de Huffman, **no es necesario implementar los algoritmos**, ya que serán provistos por la cátedra.

Ejercicio 1

Para cada imagen,

(a) Determinar empíricamente la masa de probabilidad $p(x)$ y graficarla. También calcular y comparar las entropías $H(X)$ entre las difernetes imágenes.

(b) Aplicar la compresión mediante el código de Huffman. Calcular la cantidad de bits promedio de la imagen comprimida y compararla con la longitud media teórica L del código aplicado.

Ejercicio 2

Para las mismas imágenes,

(a) Estimar la masa de probabilidad $p(y)$ de los residuos y calcular su entropía $H(Y)$. Graficar para cada imagen las funciones de probabilidad $p(x)$ y $p(y)$ para comparar las diferencias. También comparar las entropias.

(b) Codificar mediante Huffman los residuos de cada imagen y calcular la cantidad de bits promedio de los residuos comprimidos y compararla con la longitud media teórica L.

(c) Estimar empíricamente la información mutua $I(X; X')$ entre las intensidades de píxeles vecinos (considere cada par como los pares utilizados en el cálculo de residuos), siendo X la representación de un pixel y X' a de su vecino.

Ejercicio 3

A partir de los resultados de los ejercicios anteriores, responder:

(a) Comparando las entropías entre las diferentes imágenes ¿A qué se deben esas diferencias?

(b) Para la codificación diferencial mediante residuos ¿Qué está representando la información mutua calculada y a qué se deben las diferencias entre las distintas imágenes?

(c) ¿Qué diferencias observa entre la compresión pixel a pixel y la de residuos? ¿y a qué se deben esas diferencias?

Ejercicio 4 (opcional)

Implementar dos programas en python, ejecutables desde linea de comando por la temrinal.

(a) El primero que reciba como parámetro el nombre de una imagen en algún formato estándar (jpg o png) y lo guarde en binario comprimido mediante Huffman basado en residuos. El archivo debe tener extensión **.tp2**.

(b) El segundo programa, que reciba el nombre del archivo comprimido, lo descomprima y muestre la imagen.

3. Conclusiones

Como conclusiones, elabore un resumen conciso comentando las características que considere más relevantes del método estudiado y los resultados obtenidos para las distintas imágenes.

4. Apéndice

Idea básica de la códificación de Huffman

Un *código libre de prefijo* (*prefix-free*) es un tipo de codificación en el que ninguna palabra de código es prefijo de otra. Esto significa que ninguna secuencia binaria asignada a un símbolo puede coincidir con el inicio de la secuencia asignada a otro símbolo. Gracias a esta propiedad, la decodificación se puede realizar de manera *instantánea*, es decir, sin necesidad de un delimitador entre símbolos, ya que el receptor sabe exactamente cuándo termina cada palabra de código. El algoritmo de Huffman produce precisamente este tipo de códigos. Cada símbolo del alfabeto se asocia a una hoja en un árbol binario, como se muestra en la Figura 1. Para construir el código, se asigna el bit 0 a cada rama izquierda y el bit 1 a cada rama derecha (aunque esta convención puede invertirse). Este algoritmo se puede resumir en los siguientes pasos:

1. Se ordenan los símbolos por frecuencia de ocurrencia, de menor a mayor.
2. Se toman dos símbolos (o subárboles) con menor frecuencia, se combinan en un nuevo nodo cuya frecuencia es la suma de ambas.
3. El proceso se repite iterativamente, combinando las hojas o subárboles de menor frecuencia para formar un nuevo subárbol.
4. Se concluye cuando queda formado un único árbol binario.
5. Finalmente, el código binario de cada símbolo se obtiene recorriendo el árbol desde la raíz hasta la hoja correspondiente, concatenando los bits de cada rama.

Ejemplo: suponer símbolos con sus frecuencias según la Tabla 1. En la Figura 1 se muestra como quedaría armado el árbol binario y en la tabla (columna central) el resultado de la codificación final asignada a cada símbolo.

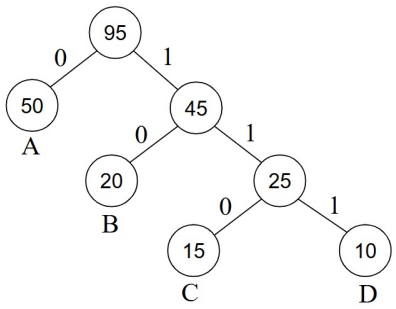


Figura 1

Símbolo	Código	Frecuencia
A	0	50
B	10	20
C	110	15
D	111	10

Tabla 1

Puede verificarse que ningún código es prefijo de otro, garantizando una codificación no ambigua. Esta estructura jerárquica es la base del **algoritmo de Huffman**, que asigna códigos más cortos a los símbolos más frecuentes y más largos a los menos frecuentes, minimizando así la longitud media del mensaje codificado.

5. Condiciones de entrega

- **Archivo ZIP**: en un archivo comprimido en formato ZIP, de nombre `TP2_GXX.zip` (donde `XX` es el número de grupo, ej: `TP2_G01.zip`) debe incluirse tanto el informe como el código fuente del trabajo. Este archivo debe subirse al recurso disponible en el campus.
- **Informe**: debe ser en formato PDF (**no se aceptarán otros formatos**) y con nombre: `TP2_GXX.pdf`. Es condición necesaria para la aprobación cumplir con las pautas para la presentación de informes estipuladas en el campus.
- **Código**: el código debe ser en un archivo de tipo cuaderno (`ipynb`). Los Ejercicios 1 y 2 pueden entregarse en cuadernos separados para mayor comodidad.

  Importante: "No debe incluirse la carpeta de imágenes entre los archivos entregados".
- Se recuerda a los estudiantes que las entregas deben ser un producto original de cada grupo, por lo que se les pide revisar la sección 6 del programa de la materia y el Código de Honor y Ética de la Universidad.

Referencias

[1] A. Vail, *The Electro-Magnetic Telegraph: A Description of the Apparatus and the Processes of Telegraphing*, New York: D. Appleton & Co., 1858.

[2] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, pp. 379–423, 623–656, 1948.

[3] R. M. Fano, "The Transmission of Information," Technical Report No. 65, Research Laboratory of Electronics, Massachusetts Institute of Technology, 1949.

[4] D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[5] J. Rissanen and G. G. Langdon, "Arithmetic Coding," *IBM Journal of Research and Development*, vol. 23, no. 2, pp. 149–162, 1979.

[6] J. Ziv and A. Lempel, "A Universal Algorithm for Sequential Data Compression," *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, 1977.

[7] J. Ziv and A. Lempel, "Compression of Individual Sequences via Variable-Rate Coding," *IEEE Transactions on Information Theory*, vol. 24, no. 5, pp. 530–536, 1978.

[8] T. Stathaki, *Methods and Standards for Lossless Compression*, Digital Image Processing, Part 4: Image Compression, Department of Electrical and Electronic Engineering, Imperial College of Science, Technology and Medicine.